Desarrollar para Gameboy: temas y referencias

Sitio:	Moodle UA 2026-27	Imprimido por:	Mingorance Boix, Víctor
Curso:	RAZONAMIENTO AUTOMÁTICO (34031)	Día:	jueves, 29 de octubre de 2026, 11:05
Libro:	Desarrollar para Gameboy: temas y referencias		

Descripción

Documentos y artículos clasificados de desarrollo y programación para Gameboy

Tabla de contenidos

1. ÍNDICE
2. Preparar el entorno
- 2.1. GBTelera: Instalación y Uso
- 2.2. Emuladores para GB
- 2.3. Ensamblador y Utiliidades

3. Desarrollar
- 3.1. Manual y TFGs de Juegos en Gameboy

3.2. Crear gráficos en GB
3.3. Temas clave para aprender
3.4. Webs de Referencia
4. Arquitectura ECS
4.1. GameEngine ECS: Starfield Effect
4.2. GameEngine ECS: Attari Assault
5. Colisiones

1. ÍNDICE

Preparar el entorno

1. GBTelera: instalación y uso
2. Emuladores de GB
3. Ensamblador y utilidades

Desarrollar

1. Manual y TFGs de Juegos en Gameboy
2. Crear gráficos en GB
3. Temas clave de aprendizaje
4. Webs de referencia

Arquitectura ECS

1. GameEngine ECS: Starfield Effect
2. GameEngine ECS: Atari Assault

Colisiones

2. Preparar el entorno

1. GBTelera: instalación y uso
2. Emuladores de GB
3. Ensamblador y utilidades

2.1. GBTelera: Instalación y Uso

GBTelera

GBTelera es un conjunto mínimo de herramientas para desarrollar cómodamente para Gameboy en ensamblador en ***Linux***. Incluye:

- Ensamblador y linker (RGBDS)
- Emuladores (BGB y Emulicious)
- Editores de tiles, sprites y tilemaps (GBTD y GBMB)
- Gestión preconfigurada de proyectos en consola con Makefile

Requiere tener instalado ***wine*** y ***java***.

Índice

- **Instalación**
- **Uso**

·· Uso ··

GBTelera se usa principalmente en el ***terminal*** de comandos.

Una vez instalada un configurada, disponemos de estos comandos:

- ***gbt_mkproject*** - Crea una carpeta con los archivos de un nuevo proyecto de ***GBTelera***.
- ***gbt_getDMGBoot*** - Descarga la BootROM original de Nintendo DMG-1 (*dmg_boot.bin*) y la copia en la carpeta de emuladores de GBTelera (<gbt_path>/tools/emulators).
- ***gbt_bgb*** - Lanza *BGB* y podemos darle como parámetro el fichero ***.gb*** para que lo carge automáticamente. (requiere ***wine***)
- ***gbt_emulicious*** - Lanza *Emulicious* y puede cargar un ***.gb*** dado como parámetro. (requiere ***java***)

Crear proyectos GBTelera

En el ***terminal***, en la carpeta donde queramos crear el proyecto, tecleamos:

> **gbt_mkproject** *<nombre-de-tu-projecto>*

esto creará la carpeta ***nombre-de-tu-proyecto***, con la estructura de subcarpetas y archivos de un proyecto de ***GBTelera*** base. Ahora sólo tienes que entrar en la carpeta y teclear:

> **make**

para ensamblar y generar la ROM (*game.gb*). Con la ROM generada ya puedes lanzar los emuladores para probarla, así:

> **gbt_bgb** *game.gb*

> **gbt_emulicious** *game.gb*

si tienes instalado ***wine*** y ***java***, estos comandos lanzarán *BGB* o *Emulicious* y cargarán la ROM *game.gb*. Esta ROM no hace nada, por lo que simplemente deberías ver salir el logo de

Nintendo y quedarse ahí parado.

Desarrollar con GBTelera

Una vez tienes un proyecto creado, tienes una carpeta con esto:

> **cfg/**
> *Makefile*
> **src/**

El fichero *Makefile* tiene la configuración básica del proyecto (su nombre y las opciones de ensamblado y linkado); y la carpeta ***cfg/*** incluye los ficheros de configuración completa del proyecto.

Para desarrollar, nos interesa la carpeta **src/**, que contiene los archivos fuente del proyecto. Inicialmente, estos dos:

> *header.asm*
> *main.asm*

Para proyectos sencillos, podemos editar *main.asm* y escribir nuestro código debajo de la etiqueta global ***main::***. Recuerda terminar con un bucle infinito o un ***halt*** (mejor si lleva ***di*** delante) para que el programa quede bloqueado.

El fichero *header.asm* tiene la información que la BootROM espera encontrar en los primeros 64 bytes (cabecera) de la ROM. Cosas como el nombre del juego, el desarrollador, el logo de Nintendo o qué recursos usa la ROM están ahí.

Si añades nuevos ficheros con extensión *.asm* en la carpeta ***src/***, serán automáticamente ensamblados y enlazados. La configuración del proyecto está diseñada para hacer esto automáticamente.

·· Instalación ··

Para usar GBTelera hay 2 opciones.

1. Repositorio de Github.
2. Máquina Virtual.

Se recomienda la opción 1, si tienes Linux. Es más cómoda, ocupa muy poco y sólo es una carpeta. Necesitarás ***wine*** y ***java***. Para Windows, la opción 2 puede ser más cómoda aunque es mucho más pesada.

GBTelera 0.1 (Github)

Conjunto mínimo de utilidades para programar en ensamblador para Gameboy.

Sólo para Linux.

Se descarga de github a una carpeta y se añade al ***path*** (instrucciones en github).

Máquina Virtual (VirtualBox)

Máquina virtual con ***Linux Manjaro***.

Trae preinstalados:

- ***GBTelera 0.1***
- ***CPCTelera 1.5***
- wine, java y compiladores de Rust y C++.

Puede ser usada en cualquier Sistema Operativo, con ***Virtualbox***.

Nota: requiere tener una CPU Intel. Si la CPU es ARM (como en los últimos Mac) hay que hacer ajustes para la virtualización.

2.2. Emuladores para GB

BGB

El más utilizado por su depurador efectivo y por la calidad de su emulación.
Recomendado como opción principal.

- Página web del emulador
- **Descarga**
  - 64 bits: **Principal** (Alternativo)
  - 32 bits: Principal (Alternativo)

Emulicius

Más actual que BGB, con algunas características de depuración especiales muy útiles.
La interfaz es más complicada y resulta incómodo para algunas tareas, pero merece la pena tenerlo para depurar en muchas circunstancias.

- Página web del emulador
- **Descarga**
  - **Principal**

Nota: Requiere JAVA para funcionar

BootROMs

Cuando la GameBoy arranca, se ejecuta un código de 256 bytes incluído dentro de la propia consola, en una ROM.
A esta ROM se le llama BootROM (ROM de arraque). También se le llama firmware.
Este código está protegido por derechos de autor y, por eso, los emuladores no suelen incluirlo

en sus descargas.
Hay que descargarlo aparte y dárselo al emulador (suelen tener una opción en su interfaz).

- Web con todas las BootROMs de GB
- **Descarga**
  - DMG_BOOT.BIN: **Principal** (Alternativo)

DMG_BOOT.BIN es el firmware de BootROM de la Game Boy Clásica original, conocida como DMG-001 (Dot Matrix Game).

2.3. Ensamblador y Utiliidades

Ensamblador: RGBDS

Ensamblador que se mantiene bastante actualizado y que permite generar ROMs en código máquina de Game Boy con interesantes utilidades para controlar ubicaciones en memoria, macros y símbolos. La sintaxis es sencilla y cómoda y está soportado por extensiones de Visual Studio Code.

- Web de RGBDS (Github)
  - Releases
  - **Versión 0.8.0 para Linux (28/jun/2024)**

Utilidades

Diseñadores de Tiles y Tilemaps (GBTD y GBMD)

Dos programas muy útiles para diseñar tiles y tilemaps para GameBoy cómodamente y luego poder exportarlos a nuestro código para incluirlos en nuestras ROMs.

- Web de GBTD/GBMD (Github)
  - Releases
  - **Versión 2.4.5 (13/jul/2021)**

3. Desarrollar

1. Manual y TFGs de Juegos en Gameboy
2. Crear gráficos en GB
3. Temas clave de aprendizaje
4. Webs de referencia

3.1. Manual y TFGs de Juegos en Gameboy

Manual de Alberto Rius

Alberto Rius Poveda elaboró su manual titulado:

- **Crea tus propios juegos para Gameboy**

, en el que os enseña paso a paso.

Además, aquí tenéis la memoria de su TFG, donde explica cómo creó el manual y algunos detalles de por qué. Os puede ser útil si queréis hacer TFGs similares:

- 2024. *Elaboración de un manual de programación en ensamblador para Game Boy* (Alberto Rius)

TFGs Previos

Estos TFGs son muy útiles para tomar ideas y aprender cómo se pueden hacer cosas en Game Boy. Todos están en ensamblador.

Además, es interesante ver hasta dónde llegaron, para tener una idea de lo que cuesta hacer juegos similares.

- 2015. **Scarlett Witch** (David Girón)
- 2018. **Galahad Escape!** (Ángel Navarro)
- 2019. **Packedtory** (Jesús Hernández)
- 2020. **Project Rogue** (Enrique Vidal)
- 2020. **Golden Sacra** (Ángel Terol)
- 2022. **Whyrm** (Víctor Conejero)

3.2. Crear gráficos en GB

- **PDF**: Cómo crear gráficos a mano (Ejemplo Roca Pokemon Rojo y Azul)

3.3. Temas clave para aprender

Lista de temas clave de aprendizaje

Usad esta lista para guiar vuestro aprendizaje y practicar los distintos temas. Os ayudará a focalizar esfuerzo en partes pequeñas que os hacen falta para conseguir tanto programar como manejar el hardware de la Gameboy lo suficiente para empezar a hacer minijuegos.

- ***Programación***
  - Cómo crear ***funciones***
    - Instrucciones relevantes: CALL, RET, PUSH y POP
  - Manejarse con ***bucles y condicionales***
    - Entender saltos condicionales (JP y JR)
    - Entender los Flags (principalmente **Z**:Zero y **CY**:Carry)
  - Cómo hacer ***cálculos***
    - Máscaras de bits (con AND, OR y XOR)
    - Sumas y restas en 8 y 16 bits (ADD, ADC, SUB, SBC, CP)
    - Multiplicaciones y divisiones por múltiplos de 2
      - Con desplazamientos (SLA, SRA)
    - Operaciones individuales de bits (BIT, SET, RES)
  - Entender ***include***, proceso de ***ensamblado***, ***enlazado*** y ***globales***
  - Usar varios ficheros de código (y entender cómo funciona todo)
  - Entender ***símbolos*** y ***etiquetas*** (DEF, EQU y EXPORT)
  - Cómo usar las ***SECTIONs*** para ubicar las cosas en ROM y RAM
- ***Gameboy***
  - Cómo borrar el VRAM Tilemap 0 (`$9800`)
  - Cómo y por qué esperar a VBLANK (Registro hardware ***rLY*** `$FF44`)
  - Qué son las paletas y cómo funcionan
  - Cómo poner la paleta del Background (Registro hardware: ***rBGP***, `$FF47`)
  - Cómo diseñar tiles a mano (VRAM Tile Data)
    - Entender la codificación binaria y crear algunos de prueba
  - Cómo añadir los datos de nuestros propios tiles a nuestra ROM
  - Cómo copiar los tiles a la VRAM Tile Data (`$8000`)
  - Cómo y por qué apagar y encender la pantalla (Registro hardware: ***rLCDC***, `$FF40`)
  - Entender el ***mapa de memoria***
  - Cómo crear un mapa a mano y copiarlo a VRAM Tilemap (`$9800`)
  - Cómo mantener en memoria posiciones y datos del personaje móvil
    - Cómo dibujarlo a partir de sus coordenadas

3.4. Webs de Referencia

Programación en Gameboy

- ***Gameboy PanDocs***
- Set de instrucciones y opcodes de Gameboy
- Gameboy CPU Manual (Nintendo)
- GBDev.io

4. Arquitectura ECS

La arquitectura Entidad-Componente-Sistema (ECS) separa los datos, la gestión de su almacenamiento y acceso, y su actualización. Esta separación sigue el principio del diseño orientado a datos (Data Oriented Design, DoD) y tiene las siguientes ventajas:

- Los datos pueden almacenarse según el patrón de acceso, maximizando la productividad del sistema (***throughput***), marcada por la memoria caché.
- La gestión de los datos está separada de su acceso y procesado.
- Favorece la componibilidad de funcionalidades, lo que permite añadirlas y quitarlas en tiempo de ejecución.
- Facilita la implementación de simulaciones.
- Simplifica el diseño de ejecución en paralelo de sistemas de procesamiento.

Por este motivo, esta arquitectura lleva más de una década siendo adoptada por la industria del videojuego y el software de simulación.

ECS Simplificado (ES)

Al no haber caché, en Gameboy es más adecuado implementar una versión simplificada de ECS, donde eliminamos la descomposición por componentes. A esta versión la llamamos ES (Entidad-Sistema).

La teoría e implementación están explicadas en vídeos, en estos subcapítulos:

- GameEngine ECS: Starfield Effect (2 vídeos)
- GameEngine ECS: Atari Assault (3 vídeos)

Tenéis también la lista completa en Youtube: ***GameEngine ECS en C***.

Diseño orientado a Datos (DoD)

Para aprender más sobre el Diseño Orientado a Datos, podéis consultar el libro gratuito ***Data Oriented Design*** de Richard Fabian.

Es un libro teórico, muy bueno y muy completo. No es necesario para la asignatura pero, si os interesa, es la mejor referencia para profundizar.

4.1. GameEngine ECS: Starfield Effect

Teoría sobre ECS (Entidad-Componente-Sistema) e implementación de un efecto de starfield.

Los vídeos se desarrollan en C para Amstrad CPC. La idea es que podáis aprender aspectos teóricos y prácticos de cómo funciona un Motor ECS. En esta serie aprendemos qué son entidades y cómo se gestionan, qué es crearlas y destruirlas y cómo implementar todo.

- GameEngine ECS: Starfield Effect (1 / 2)
  - Ejemplo de Starfield (Campo de Estrellas)
  - ***Qué es un ECS*** simplificado (ES)
  - ***Entidades***: gestión, estructura
  - ***Sistemas***: procesado de entidades
  - Implementación básica en C: arrays y flags
  - Memoria y optimizaciones en 8-bits
  - ***Depuración*** y manejo de errores
  - Render básico
- GameEngine ECS: Starfield Effect (2 / 2)
  - Resumen de estado y plan de acción
  - Precondiciones y postcondiciones en código
  - ***Inversión de control***
  - Entidades, tipos y máscaras
  - ***Destrucción*** (borrado) de entidades
  - Creación dinámica y pseudo-aleatoriedad
  - ***Generación*** de entidades

4.2. GameEngine ECS: Attari Assault

Implementación del juego Atari Assault en C para Amstrad CPC con una Arquitectura ECS (ES) más avanzada que el Starfield Effect.

Se añade diferenciación de componentes de entidades, gráficos, gestor de juego, colisiones, reacciones a colisiones, comportamientos de IA y gestión más avanzada de la memoria.

- GameEngine ECS: Atari Assault (1 / 3)
  - Presentación de Atari Assault
  - Revisión del starfield anterior y propuesta de cambios
  - **Componentes** de entidades
  - ***for_all_matching*** (selección por componente y firma)
  - Render usando sprites
  - Manager de Juego
- GameEngine ECS: Atari Assault (2 / 3)
  - Revisión del juego y plan de mejoras
  - ***Polimorfismo*** con ***punteros a función*** (vs switch)
  - Plantillas de entidades
  - ***Comportamientos*** de enemigos
  - Preparativos para añadir ***animaciones***
- GameEngine ECS: Atari Assault (3 / 3)
  - Revisión del juego y plan de acción
  - Inicialización estática, constantes y punteros
  - Como inspeccionar binarios y ***depurar***
  - ***Clonado y borrado*** de entidades; entidades fantasma

- Refactorización
- ***Colisiones***, ***capas, bounding boxes*** y pares
- Programación: East/West const, macros, expresiones constantes, optimización

5. Colisiones

Colisiones con el tilemap

Explicaciones de cómo calcular colisiones de sprites contra un tilemap. Requiere calcular posiciones en tiles y acceder a la matriz (array) del tilemap y comprobar qué tile hay. Estos vídeos explican los cálculos y ayudan a visualizar lo que hay que hacer y cómo funciona:

- [2021] Cómo calcular colisiones con el tilemap [#Z80 #ASM]
  - Cómo crear un sistema de colisiones ECS.
  - Explicación teórica del cálculo de colisiones.
  - Realizar los cálculos en ensamblador Z80 para obtener las posiciones de memoria dentro del tilemap.
- [2022] Cálculos y colisiones con el tilemap ***(Vídeo Privado)***
  - Explicación teórica más pausada (más paso a paso)
  - Visualización de la memoria y tilemaps como arrays
  - ¿En qué consisten exactamente las colisiones con el tilemap?
  - Calcular posiciones en tiles y direcciones de memoria
  - Optimizar los cálculos
  - Detectar la colisión y deshacerla

Importante: ambos vídeos están hechos en ensamblador Z80 para Amstrad CPC. Si bien es similar a lo que hay que hacer en Gameboy, no es lo mismo. Quedaos con las explicaciones y cread el código para Gameboy. En Gameboy no tenéis registros IX ni IY, ni tampoco tenéis que usar las funciones de CPCtelera, ni tampoco podéis leer los píxeles de la pantalla.